

Quiz 03: Dynamic Arrays and Amortized Analysis

Analysis Document – The Snake Challenge

Estructuras de Datos – Universidad Nacional de Colombia

Laura Victoria Flórez Forero
Juan Sebastian Manrique Mesa
Jhon Alexander Berdugo Pasachoa

27 de septiembre de 2026

1. Representación Interna y Modelo de Estado (Pregunta 3)

Para resolver la ineficiencia de los desplazamientos lineales ($\Theta(k)$), la clase **Snake** se implementa utilizando la técnica de **Arreglo Circular (Ring Buffer)** sobre un arreglo nativo de Java (`Position[]`).

1.1. Variables de Estado

Para garantizar un estado mínimo no redundante en memoria, la estructura almacena explícitamente:

- **data**: Arreglo nativo de tipo `Position[]` que almacena los elementos físicos de la serpiente.
- **tail**: Índice físico (*int*) en **data** donde reside la cola de la serpiente (p_0).
- **size**: Cantidad de elementos lógicos activos actualmente en la serpiente (*int*).
- **capacity**: Tamaño físico total del arreglo nativo **data** (*int*).

Nota sobre la Cabeza (head): La ubicación de la cabeza no requiere una variable de estado almacenada explícitamente. Se calcula de forma derivada en $O(1)$ cuando $\text{size} > 0$ mediante:

$$\text{physicalHeadIndex} = (\text{tail} + \text{size} - 1) \bmod \text{capacity} \quad (1)$$

Asimismo, la posición receptora para la inserción de una nueva cabeza en `addHead()` se calcula en $O(1)$ como $(\text{tail} + \text{size}) \bmod \text{capacity}$.

1.2. Transformación Formal de Índices

Dado un índice lógico $i \in [0, \text{size} - 1]$, donde $i = 0$ representa la cola (p_0) e $i = \text{size} - 1$ representa la cabeza (p_{k-1}), su posición física correspondiente dentro del arreglo se calcula mediante:

$$\text{physicalIndex}(i) = (\text{tail} + i) \bmod \text{capacity} \quad (2)$$

1.3. Procedimiento de Resize

Cuando $\text{size} == \text{capacity}$, la operación `addHead()` duplica la capacidad ($\text{capacity}_{\text{new}} = 2 \cdot \text{capacity}_{\text{old}}$). En el procedimiento `resize()`, los elementos se desenrollan y reordenan de forma contigua en el nuevo arreglo manteniendo el orden lógico exacto desde $i = 0$ hasta $i = \text{size} - 1$, fijando nuevamente $\text{tail} = 0$.

2. Invariantes del Sistema

2.1. Invariantes de `DynamicArray<T>`

1. **Límites de Capacidad y Tamaño**: $0 \leq \text{size} \leq \text{capacity}$ y $\text{capacity} \geq 1$ en todo momento.
2. **Contigüidad Física**: Los size elementos lógicos residen secuencialmente en las posiciones físicas $0, 1, \dots, \text{size} - 1$.
3. **Ausencia de Memory Leaks (Loitering)**: $\forall j \in [\text{size}, \text{capacity} - 1], \text{data}[j] == \text{null}$.

2.2. Invariantes de Snake

1. **Relación Lógica-Física:** $\forall i \in [0, \text{size} - 1]$, el elemento lógico i está guardado en $\text{data}[(\text{tail} + i) \bmod \text{capacity}]$.
2. **Ubicación de la Cabeza Lógica:** Si $\text{size} > 0$, el elemento correspondiente a la cabeza ($p_{\text{size}-1}$) se encuentra siempre en la posición física $(\text{tail} + \text{size} - 1) \bmod \text{capacity}$.
3. **Aislamiento de Celdas Libres:** Toda posición física fuera del rango cíclico activo de la serpiente contiene estrictamente `null`.

3. Análisis de Complejidad Temporal

3.1. Análisis Peor Caso (Worst-Case) y Estructuras Básicas (Pregunta 1)

- **get() y set():** Complejidad $O(1)$, pues el acceso por índice en un arreglo nativo toma tiempo constante.
- **removeLast():** Complejidad $O(1)$, ya que solo requiere decrementar la variable `size` y limpiar la última referencia.
- **append():** Complejidad $O(n)$ en el peor caso debido a que cuando $\text{size} = \text{capacity}$, el redimensionamiento obliga a reubicar secuencialmente los n elementos al nuevo bloque de memoria.

*Justificación de la Amortización en **append()**:* El costo peor caso $O(n)$ no contradice su tiempo amortizado constante $O(1)$. Como el redimensionamiento ocurre exclusivamente cuando la memoria se agota y se duplica la capacidad, transcurren al menos n inserciones $O(1)$ antes del siguiente llenado. Esto permite distribuir matemáticamente el costo de la copia entre las inserciones previas, promediando $O(1)$ por operación.

3.2. Análisis de la Implementación Ineficiente (Pregunta 2)

- **a) Complejidad de REMOVE_TAIL():** Es $\Theta(k)$, donde $k = \text{size}$. Fijar la cola en el índice 0 obliga a ejecutar un bucle de $k - 1$ desplazamientos hacia la izquierda para cubrir la vacante dejada por la cola.
- **b) Complejidad total de m movimientos sin comer:** Cuesta $\Theta(m \cdot k)$. Cada turno ejecuta un `addHead()` ($O(1)$) seguido de un `REMOVE_TAIL()` ($\Theta(k)$). La suma acumulada para m pasos es:

$$T(m, k) = \sum_{i=1}^m \Theta(k) = \Theta(m \cdot k) \quad (3)$$

- **c) Validez de la afirmación del estudiante:** La afirmación es falsa. Aunque la inserción aislada posea un costo amortizado bajo, el movimiento `MOVE()` acopla obligatoriamente la eliminación frontal. El costo del movimiento queda dominado por la operación más costosa:

$$T_{\text{MOVE}} = T_{\text{addHead}} + T_{\text{REMOVE_TAIL}} = O(1) + \Theta(k) = \Theta(k) \quad (4)$$

Un costo de inserción amortizado constante no cancela ni mitiga el costo lineal estricto $\Theta(k)$ que ejecuta obligatoriamente `REMOVE_TAIL()` en cada avance sin comer.

- **d) Cuello de Botella (Bottleneck):** El cuello de botella principal es el acoplamiento rígido entre la cola lógica (p_0) y la posición física 0 del arreglo (`data[0]`). Fijar la cola en el origen destruye la flexibilidad de la estructura, forzando un desplazamiento masivo de memoria que vuelve inviable el algoritmo para serpientes de gran longitud.

4. Demostración Formal del Análisis Amortizado (Pregunta 4)

4.1. Método del Agregado (Aggregate Method)

Sea m el número total de movimientos y $g \leq m$ el número de manzanas consumidas.

- **Longitud Máxima Transitoria ($N_{\text{máx}}$):** Iniciando con $\text{size} = 2$, en un paso sin comer `addHead()` incrementa transitoriamente el tamaño antes de que `removeTail()` lo reduzca. Con g manzanas, la longitud máxima transitoria alcanzada es:

$$N_{\text{máx}} = 2 + g + 1 = g + 3 \quad (5)$$

- **Copias Totales en Redimensionamientos:** Los redimensionamientos ocurren en capacidades $2, 4, 8, \dots, 2^k$, donde $2^k \leq N_{\text{máx}} < 2^{k+1}$. Las copias totales están acotadas por:

$$\text{Total Copias} = \sum_{j=1}^k 2^j = 2^{k+1} - 2 < 2N_{\text{máx}} = 2(g+3) = O(g) \quad (6)$$

- **Costo Total y Costo Amortizado:** En m movimientos se realizan m escrituras, $m - g$ eliminaciones $O(1)$ y $O(g)$ copias por `resize`:

$$T(m) = O(m) + O(m - g) + O(g) = O(m) \implies \hat{c}_{\text{addHead}} = \frac{O(m)}{m} = O(1) \quad (7)$$

4.2. Método del Banco / Contabilidad (Banker's Method)

Asignamos una tarifa amortizada (crédito) de $\hat{c} = 3$ fichas por cada operación `addHead()`:

- 1 ficha paga la inserción física inmediata del nuevo elemento.
- 1 ficha se guarda como ahorro personal asignado a dicho elemento recién insertado.
- 1 ficha se guarda como ahorro para subsidiar el traslado de un elemento antiguo posicionado previamente en el arreglo.

Análisis de Sufficiencia: Tras una expansión a capacidad C , la estructura ha procesado exactamente $C/2$ nuevas inserciones desde el redimensionamiento anterior (cuando la capacidad era $C/2$). Esas $C/2$ inserciones acumularon exactamente $\frac{C}{2} \times 2 = C$ fichas de ahorro. Cuando el arreglo alcanza su límite de capacidad C , las C copias requeridas para trasladar los elementos al nuevo arreglo de capacidad $2C$ son pagadas íntegramente por los C créditos acumulados. Dado que `removeTail()` ejecuta la liberación en $O(1)$ sin modificar la capacidad, los fondos ahorrados nunca se pierden.

4.3. Método del Potencial (Potential Method)

Definimos la función potencial sobre el estado del arreglo circular como:

$$\Phi = 2 \cdot \text{size} - \text{capacity} \quad (8)$$

Dado que $\text{capacity}/2 \leq \text{size} \leq \text{capacity}$, se garantiza que $\Phi \geq 0$, con $\Phi_0 = 2(2) - 2 = 2 \geq 0$.

1. **addHead() sin Resize (size < capacity):** $c_i = 1$, $\Delta\Phi = 2(\text{size} + 1) - \text{capacity} - (2\text{size} - \text{capacity}) = 2$. $\hat{c}_i = c_i + \Delta\Phi = 1 + 2 = 3 = O(1)$.
2. **addHead() con Resize (size == capacity = C):** $c_i = C + 1$ (C copias + 1 inserción). $\Phi_{i-1} = C$. Tras el `resize` ($\text{capacity} = 2C$, $\text{size} = C + 1$), $\Phi_i = 2(C + 1) - 2C = 2$. $\Delta\Phi = 2 - C$. $\hat{c}_i = (C + 1) + (2 - C) = 3 = O(1)$.
3. **removeTail() ($O(1)$ worst-case):** $c_i = 1$, $\Delta\Phi = -2$. $\hat{c}_i = 1 - 2 = -1 \leq 0$.

5. Análisis del Escenario Adversario (The Adversarial Snake)

Un adversario que intente alterar la secuencia de movimientos (intercalando giros, pausas o patrones irregulares de consumo de manzanas) **no puede degradar la complejidad amortizada de $O(1)$ a peor caso $O(n)$** .

La razón fundamental es que la estructura opera bajo un régimen libre de contracciones de memoria (*downsizing*); la capacidad del arreglo nativo crece de manera estrictamente monótona. Los movimientos vacíos (sin comer) se limitan a avanzar los punteros `tail` y `head` mediante aritmética modular en $O(1)$ sin agotar ni aumentar el espacio ocupado. Para provocar un comportamiento degenerativo o un fenómeno de *thrashing* (redimensionamientos continuos de alto costo), el adversario requeriría la capacidad de forzar reducciones de tamaño en el límite de la capacidad, lo cual no ocurre en el modelo del juego.

6. Evaluación de Políticas de Crecimiento (Pregunta 5)

6.1. Tabla Comparativa de Políticas

Política	# Resizes	Total copias	Costo amortizado por crecimiento	Espacio sin usar tras resize
$C + 1$ (A)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n)$	0 (0%)
$2C$ (B)	$\Theta(\log n)$	$\Theta(n)$	$\Theta(1)$	$\approx n/2$ (50 %)
$1,5C$ (C)	$\Theta(\log_{1,5} n)$	$\Theta(n)$	$\Theta(1)$	$\approx n/3$ (33,3 %)

6.2. Demostración para la Familia General $C_{\text{new}} = \lceil \alpha C \rceil$

Para cualquier constante fija $\alpha > 1$, las capacidades forman una progresión geométrica $C_i = C_0 \cdot \alpha^i$. El total de elementos copiados al alcanzar la longitud n es:

$$\text{Total Copias} = \sum_{i=0}^{k-1} C_i = C_0 \frac{\alpha^k - 1}{\alpha - 1} < \frac{C_k}{\alpha - 1} \leq \frac{\alpha}{\alpha - 1} \cdot n = \Theta(n) \quad (9)$$

Dividiendo entre n operaciones, el costo amortizado por crecimiento es:

$$\hat{c} = \frac{\Theta(n)}{n} = \Theta(1) \quad \forall \alpha > 1 \quad (10)$$

Esto demuestra que no es estrictamente obligatorio duplicar la capacidad ($\alpha = 2$); cualquier factor multiplicativo constante $\alpha > 1$ garantiza costo amortizado constante $O(1)$.